



Vita-Link Admin

Documentation Technique du Projet

Stack	Next.js 15 · TypeScript · Tailwind · shadcn/ui
API Backend	vita-link-api.onrender.com/api
Gestion d'état	Zustand · TanStack Query
Auth	NextAuth.js (JWT)
Version	0.1.0 — Initial Setup

1. Architecture du projet

Le projet suit une architecture en couches stricte inspirée des principes SOLID. Chaque couche a une responsabilité unique et ne communique qu'avec la couche immédiatement en dessous.

1.1 Vue d'ensemble des couches

Couche	Dossier	Rôle
Pages	src/app/	Routes Next.js (App Router) — ce que l'utilisateur voit
Composants	src/components/	Blocs UI réutilisables organisés par module
Hooks	src/lib/hooks/	Logique métier — combinent React Query + services
Store	src/store/	État global Zustand (filtres, sidebar)
Services	src/services/	Appels API — un fichier par ressource
Client HTTP	src/lib/api/	Axios centralisé — token, erreurs, redirect 401
Types	src/types/	Tous les types TypeScript du domaine métier
Constantes	src/lib/constants/	Régions, groupes sanguins, grades Jambaar

1.2 Arborescence complète

Voici la structure de fichiers du projet avec le rôle de chaque élément :

vita-link-admin/	
├─ src/	
│ └─ app/	# Next.js App Router
│ └─ (auth)/login/	# Page de connexion
│ └─ (dashboard)/	# Pages protégées (auth requise)
│ └─ dashboard/	# Vue hélicoptère + KPIs
│ └─ structures/	# Certification des hôpitaux
│ └─ jambaars/	# Modération des donneurs
│ └─ rewards/	# Régie Jambaar Life
│ └─ reports/	# Statistiques & export
│ └─ layout.tsx	# Sidebar + Header partagés
│ └─ globals.css	# Styles Tailwind + variables CSS
│ └─ layout.tsx	# Root layout (Providers)
│ └─ components/	
│ └─ ui/	# Composants shadcn/ui (auto-générés)
│ └─ layout/	# Sidebar, Header, Providers
│ └─ dashboard/	# KPICard, AlertsPanel, RegionHeatmap
│ └─ structures/	# StructuresTable
│ └─ jambaars/	# JambaarDirectory
│ └─ rewards/	# PartnersTab, BadgesTab
│ └─ reports/	# DonationTrendChart, ExportPanel
│ └─ shared/	# FilterBar, StatusBadge
└─ services/	# Un fichier = une ressource API

```

├── dashboard.service.ts
├── structures.service.ts
├── jambaars.service.ts
├── rewards.service.ts
├── lib/
│   ├── api/client.ts          # Client HTTP centralisé (axios)
│   ├── hooks/                 # Custom hooks React
│   ├── utils/                 # Fonctions utilitaires
│   └── constants/             # Données métier fixes
├── types/index.ts             # Types TypeScript globaux
├── store/                     # État global Zustand
├── .github/
│   ├── workflows/ci.yml       # CI/CD automatique
│   └── PULL_REQUEST_TEMPLATE/ # Template PR
├── .env.example               # Variables à copier
├── .env.local                  # Variables réelles (jamais committé)
└── .npmrc                     # legacy-peer-deps=true

```

1.3 Principes SOLID appliqués

Principe	Nom	Application dans Vita-Link Admin
S	Single Responsibility	1 service = 1 ressource API. 1 composant = 1 responsabilité.
O	Open/Closed	Composants extensibles via props sans modification directe.
L	Liskov	Sous-composants interchangeables (ex: différents types de charts).
I	Interface Segregation	Types TypeScript granulaires, jamais de any.
D	Dependency Inversion	Les pages dépendent des hooks, les hooks des services, jamais d'axios directement.

2. GitHub — Configuration et workflow

2.1 Initialisation du dépôt

Après avoir créé le repo sur GitHub, exécuter ces commandes dans l'ordre :

```
# 1. Initialiser git et pousser le code
git init
git add .
git commit -m "chore: initial project setup"
git remote add origin https://github.com/[USERNAME]/vita-link-admin.git
git push -u origin main

# 2. Créer et pousser la branche develop
git checkout -b develop
git push -u origin develop
```

2.2 Inviter un collaborateur

- Aller sur le repo GitHub → Settings → Collaborators and teams
- Cliquer sur Add people et saisir le nom d'utilisateur GitHub
- Choisir le rôle : Write (peut push) ou Maintain (peut merger les PRs)
- Le collaborateur reçoit un email d'invitation à accepter

2.3 Branch Protection Rules

Ces règles empêchent les push directs sur les branches principales. À configurer dans Settings → Branches → Add branch protection rule.

Pour main ET develop, cocher :

- Require a pull request before merging
- Require approvals : 1 (au minimum un reviewer)
- Require status checks to pass — ajouter le job CI : Lint + Type-check + Build
- Require branches to be up to date before merging
- Do not allow bypassing the above settings

2.4 Secrets GitHub Actions

Aller dans Settings → Secrets and variables → Actions → New repository secret :

Nom du secret	Valeur
NEXT_PUBLIC_API_URL	https://vita-link-api.onrender.com/api
NEXTAUTH_SECRET	Générer avec : openssl rand -base64 32

3. Branches et nommage

3.1 Branches principales

Branche	Protection	Rôle
main	PR + 1 review + CI	Production — code déployé
develop	PR + 1 review + CI	Intégration — toutes les features convergent ici
feat/*	Libre	Nouvelle fonctionnalité
fix/*	Libre	Correction de bug
chore/*	Libre	Tâche technique (dépendances, config)
docs/*	Libre	Documentation uniquement
hotfix/*	Libre	Correctif urgent en production

3.2 Convention de nommage

Format : <type>/<scope>-<description-courte>

```
feat/dashboard-heatmap
feat/structures-validation-modal
fix/auth-token-refresh
chore/upgrade-nextjs-15
docs/api-integration-guide
hotfix/critical-login-bug
```

3.3 Flux de travail

```
main  ← develop  ← feat/ma-feature
      ← fix/mon-bug
      ← chore/ma-tache
```

4. Push et Pull Requests

4.1 Workflow quotidien

```
# 1. Toujours partir de develop à jour
git checkout develop
git pull origin develop

# 2. Créer sa branche
git checkout -b feat/mon-module-ma-feature

# 3. Coder, committer souvent
git add .
git commit -m "feat(dashboard): add blood group filter to KPI cards"

# 4. Pousser
git push origin feat/mon-module-ma-feature

# 5. Ouvrir une PR sur GitHub : feat/... → develop
```

4.2 Format des commits — Conventional Commits

Format : <type>(<scope>): <description>

Type	Usage
feat	Nouvelle fonctionnalité
fix	Correction de bug
chore	Tâche technique (dépendances, config, CI)
docs	Documentation uniquement
refactor	Refactoring sans changement de comportement
perf	Amélioration de performance

Exemples de commits valides :

```
feat(structures): add document validation modal
fix(auth): handle expired JWT token gracefully
chore(deps): upgrade tanstack-query to v5
refactor(dashboard): extract KPI logic to custom hook
```

4.3 Checklist avant d'ouvrir une PR

- npm run lint — zéro erreur ESLint
- npm run type-check — zéro erreur TypeScript
- npm run build — build réussi sans avertissement
- Pas de any dans le code TypeScript
- Pas de .env.local committé

- La PR cible develop (jamais main directement)
- Au moins un reviewer assigné

5. shadcn/ui

5.1 Principe fondamental

shadcn/ui n'est pas une librairie npm classique. Chaque composant est copié directement dans le projet dans `src/components/ui/`. Vous en êtes propriétaire et pouvez le modifier.

5.2 Ajouter un composant

```
npx shadcn@latest add [nom-du-composant]

# Exemples :
npx shadcn@latest add dialog
npx shadcn@latest add table
npx shadcn@latest add input
npx shadcn@latest add form
```

Si une erreur de peer deps apparaît, ajouter `--legacy-peer-deps` au fichier `.npmrc` :

```
echo 'legacy-peer-deps=true' > .npmrc
```

5.3 Composants déjà installés

Composant	Usage dans le projet
button	Boutons d'action partout dans l'interface
card	Cartes KPI, cartes Jambaar, cartes partenaires
skeleton	États de chargement (loading states)
tabs	Navigation entre onglets (ex: Partenaires / Badges)
select	Filtres région et groupe sanguin
badge	Étiquettes de statut (Actif, Suspendu...)
avatar	Initiales des Jambaars dans les cartes
dropdown-menu	Menus d'actions contextuelles (Valider, Rejeter...)

5.4 Règles importantes

- Ne jamais modifier les fichiers dans `src/components/ui/` — ils peuvent être régénérés par shadcn
- Toujours choisir Radix puis Custom lors de l'initialisation
- Le fichier `components.json` doit exister avant d'ajouter des composants
- Référence complète : ui.shadcn.com/docs/components

6. Setup complet pour un nouveau développeur

6.1 Prérequis

- Node.js >= 20.x
- npm >= 10.x
- Git configuré avec votre nom et email
- Invitation GitHub acceptée (envoyée par le lead)

6.2 Installation

```
# 1. Cloner le repo
git clone https://github.com/[ORG]/vita-link-admin.git
cd vita-link-admin

# 2. Configurer npm (évite les erreurs de dépendances)
echo 'legacy-peer-deps=true' > .npmrc

# 3. Installer les dépendances
npm install

# 4. Créer le fichier d'environnement
cp .env.example .env.local
# → Ouvrir .env.local et remplir NEXTAUTH_SECRET

# 5. Lancer le projet
npm run dev
# → Ouvrir http://localhost:3000
```

6.3 Variables d'environnement requises

Variable	Description
NEXT_PUBLIC_API_URL	URL de l'API backend Vita-Link
NEXTAUTH_SECRET	Clé secrète pour NextAuth.js (openssl rand -base64 32)
NEXTAUTH_URL	URL de l'application (http://localhost:3000 en dev)

7. Règles de collaboration

7.1 À faire

- Toujours partir de develop avant de créer une branche
- Committer souvent avec des messages Conventional Commits clairs
- Ouvrir une PR par fonctionnalité (pas plusieurs features dans une PR)
- Demander une review avant de merger
- Mettre à jour .env.example si vous ajoutez une variable d'environnement
- Écrire des types TypeScript précis — jamais de any

7.2 À ne jamais faire

- Push direct sur main ou develop — toujours passer par une PR
- Committer .env.local — ce fichier contient des secrets
- Utiliser any en TypeScript — cela casse la sécurité des types
- Modifier src/components/ui/ — ces fichiers sont gérés par shadcn
- Appeler axios directement dans un composant — passer par les services
- Merger sans que le CI soit vert — lint, type-check et build doivent passer

7.3 Où mettre son code

Ce que je crée	Où le mettre
Nouvelle page	src/app/(dashboard)/[module]/page.tsx
Composant d'un module	src/components/[module]/MonComposant.tsx
Composant réutilisable	src/components/shared/MonComposant.tsx
Appel API	src/services/[resource].service.ts
Hook custom	src/lib/hooks/useMonHook.ts
Type TypeScript	src/types/index.ts
Constante métier	src/lib/constants/index.ts
État global	src/store/[nom].store.ts

*"L'interface Admin de Vita-Link est un outil de santé publique. Elle permet à l'État et au CNTS de voir l'invisible : **le mouvement du sang dans tout le pays en temps réel.**"*